Revision — 2.1.2 Thinking Ahead

OCR H446 — one-page revision summary

Must know

- **Thinking ahead** = planning before coding: identifying inputs/outputs, preconditions, what to cache and which reusable components to use.
- **Inputs / outputs:** data *in* vs results *out*. "Validate the data" is a **process**, not an input or output — a common error.
- **Preconditions:** a condition that must be **true before** a routine runs correctly (e.g. a list must be **sorted before** a binary search; a value must be non-zero before dividing).
- **Caching:** store expensive or frequently used results in fast storage for reuse. **Benefit:** faster, less recomputation/network traffic. **Trade-off:** extra storage, and data can become **stale** if the source changes → needs invalidation.
- **Reusable components:** self-contained, generalised modules used across a program/projects. **Benefits:** faster development, fewer bugs (pre-tested), consistency, easier maintenance.
- Thinking ahead reduces wasted work, catches problems early and produces designs that are efficient and maintainable.

Key terms

Term	Definition
Input	Data supplied to a process
Output	Result produced by a process
Precondition	A condition that must be true before a routine runs correctly
Caching	Storing expensive/frequent results in fast storage for reuse
Stale data	Cached data that is out of date because the source changed
Cache invalidation	Removing/refreshing stale cached data
Reusable component	Self-contained, generalised module usable in many places

Worked example / how to — apply thinking ahead to an online cinema booking system

- **Inputs:** film choice, date/time, number of seats, payment details. **Outputs:** seat allocation, booking confirmation/reference, updated seat map. (Note: "validate card details" is a *process*.)
- **Precondition:** the requested seats must still be available before the booking is confirmed — check `seatsRequested <= seatsAvailable` first.
- **Caching:** cache the current seat map / showtimes so repeat page loads don't re-query the database every time — **but** invalidate the cache when a booking is made so it isn't **stale**.
- **Reusable components:** reuse a generic payment module and a date-validation routine already written and tested for other pages, speeding development and reducing bugs.

Exam tips

- **Apply, don't define** — list the scenario's *actual* inputs/outputs, precondition, cache target and reusable modules.
- Never call a process (validating, sorting, calculating) an input or output.
- For caching, always **pair the benefit with the trade-off** (storage cost / stale data).
- State a precondition as something that must be true **before** the routine runs.